

STARWU-LITE 开发框架

软件设计说明书

1 引言

1.1 软件简介

【STARWU-LITE 开发框架 V1.0】是一款基于 Java 语言自研的轻量级开发框架。

本框架主要面向 Java 后端开发场景，使一个相对简陋的项目，通过集成该框架，成为一个非常成熟的项目。

本框架运行依赖 JDK 环境，不包含可视化 GUI 界面，以 Jar 包形式给业务项目引入使用。

1.2 开发目的

自从 SpringBoot 框架问世以后，受到了很多 java 开发人员的欢迎，其中一个很重要的原因就是它简化了很多的配置，使我们可以快速的搭建一个项目。我们只要用 maven 把 SpringBoot 的 jar 包引入进来，然后在写个启动类，在写一个 Controller，就可以启动项目，并且对外提供服务了。

但是问题是，这样的项目还是一个很简陋的项目，还远远不够成熟，还有很多很多的工作需要我们去处理，比如：请求日志如何记录、响应参数如何封装、异常怎样统一处理、threadLocal 对象如何清除、数据库与 redis 如何配置使用、怎样避免 redis 的缓存击穿、缓存穿透问题、怎么处理 redis 与数据库的数据一致性的问题，如果用到分布式了，分布式事务、分布式 session 怎么解决、如何监控慢 sql，等等等等，这些问题都需要我们去处理。

这些问题看起来都不是特别难以处理，但是如果每个问题都需要我们去自己处理的话，也是一件很麻烦的事情。

而本框架，就是为了去解决这些问题，只要在刚刚搭建的 SpringBoot 项目中，引入本框架的 jar 包，就可以将一个很简陋的项目，升级到一个足够成熟的项目，包括且不限于上面提到的问题，都在框架中帮助我们处理了，所以，我们就可以利用本框架，快速的搭建起一个很成熟的项目。

本软件著作权保护范围为本项目自研编写的全部源代码，框架运行所依赖的 JDK、Maven 以及其他第三方开源组件，不属于本软件著作权保护范围。

1.3 应用领域

本软件适用于 Java 后端服务开发、小型服务项目等场景。

1.4 运行环境

硬件环境

CPU: Intel/AMD 双核及以上;

内存: 4GB 及以上;

硬盘: 100MB 以上可用存储空间。

软件环境

开发语言: Java

JDK 版本: JDK8

构建工具: Maven 3.3.3

运行操作系统: Windows10/11、Linux (CentOS7、Ubuntu)

2 软件总体架构

2.1 整体架构

本框架采用模块化设计，整体分为以下几个模块：

1. **START 模块：**框架的统一入口。
2. **WEB 模块：**主要负责对 web 请求做一些封装，比如记录日志、统一异常处理、响应参数的封装等等。
3. **DAO 模块：**基于 mybatisPlus，封装了 mybatisPlus 与 redis 的结合使用得过程，以及处理 redis 与数据库数据一致性的问题，和 redis 缓存击穿、缓存穿透等问题。和监控慢 sql，以及记录数据库变更日志等功能。
4. **CANAL 模块：**通过集成阿里的 canal 中间件，监听数据库的所有变更记录，并且推送给所有的订阅者。订阅者可以依据此记录来解决一系列问题，比如数据一致性的问题、记录数据库变更记录等等等等。
5. **DISTRIBUTED 模块：**负责解决分布式带来的一系列问题，目前解决了分布式 session 和分布式事务的问题。
6. **DESIGN 模块：**负责对设计模式做一些封装，目前封装了工厂模式、责任链模式、观察者模式。
7. **MANAGE 模块：**负责对一对象资源做统一管理，比如 ThreadLocal 对象的统一管理，Spring 的异步资源的统一管理等等。
8. **METADATA 模块：**定义各个模块都会用到的一些元数据信息，比如：注解、枚举、实体类、配置信息等等等等。

9. UTIL 模块：定义一些常用的工具类。

2.2 架构图

该框架的架构图如下：

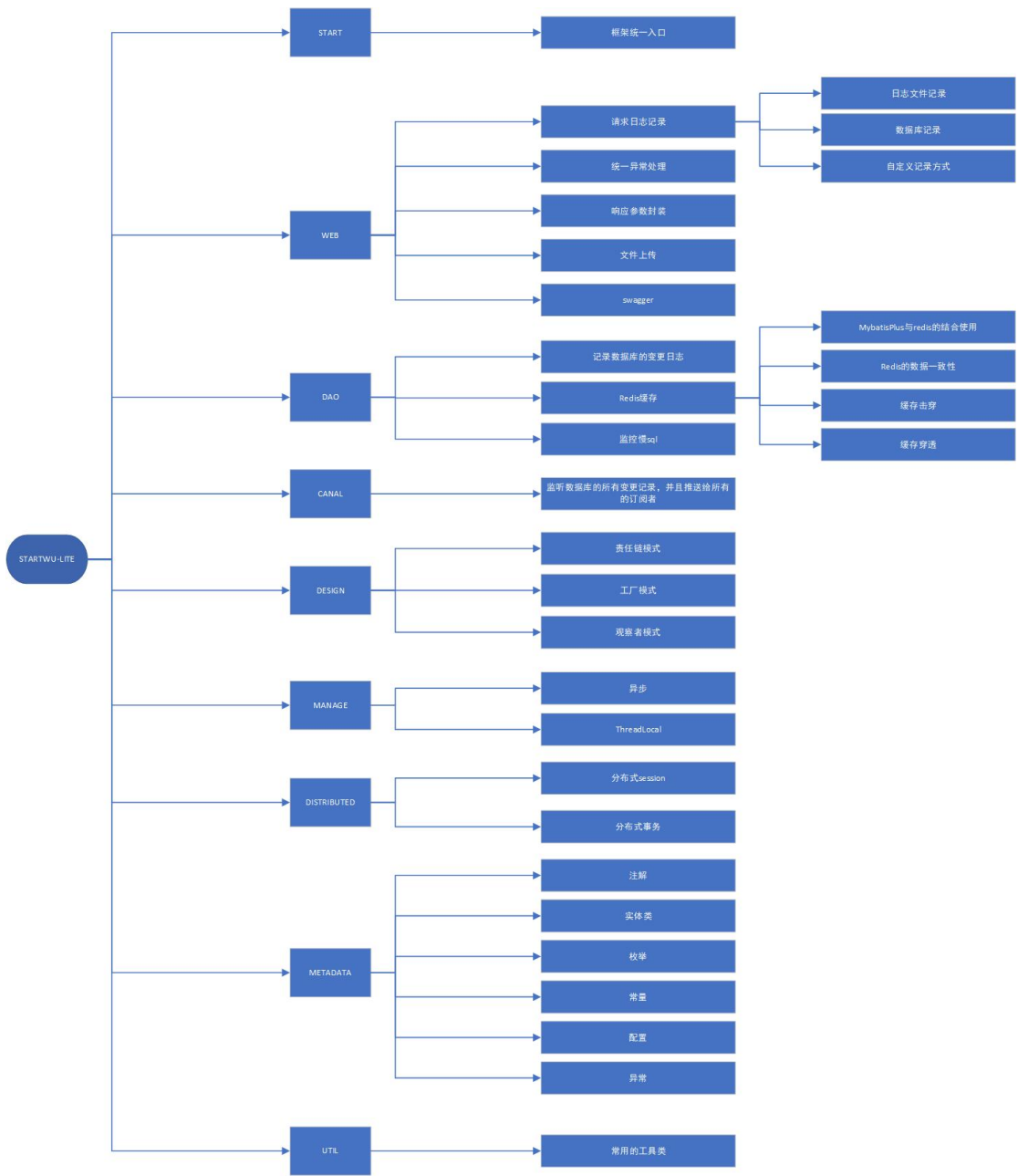


图 2-1 软件架构图 1

3 详细设计说明

3.1 整体设计

3.1.1 框架引入

本框架采用 maven 模块化开发，根据对外提供的功能的不同，来划分不同的 module，具体的划分规则，已经在上文中描述。

其中 START 模块是该框架的入口，在该模块中，没有任何的代码，它只是把其他所有的模块都依赖进来，如下图 3-1 所示。

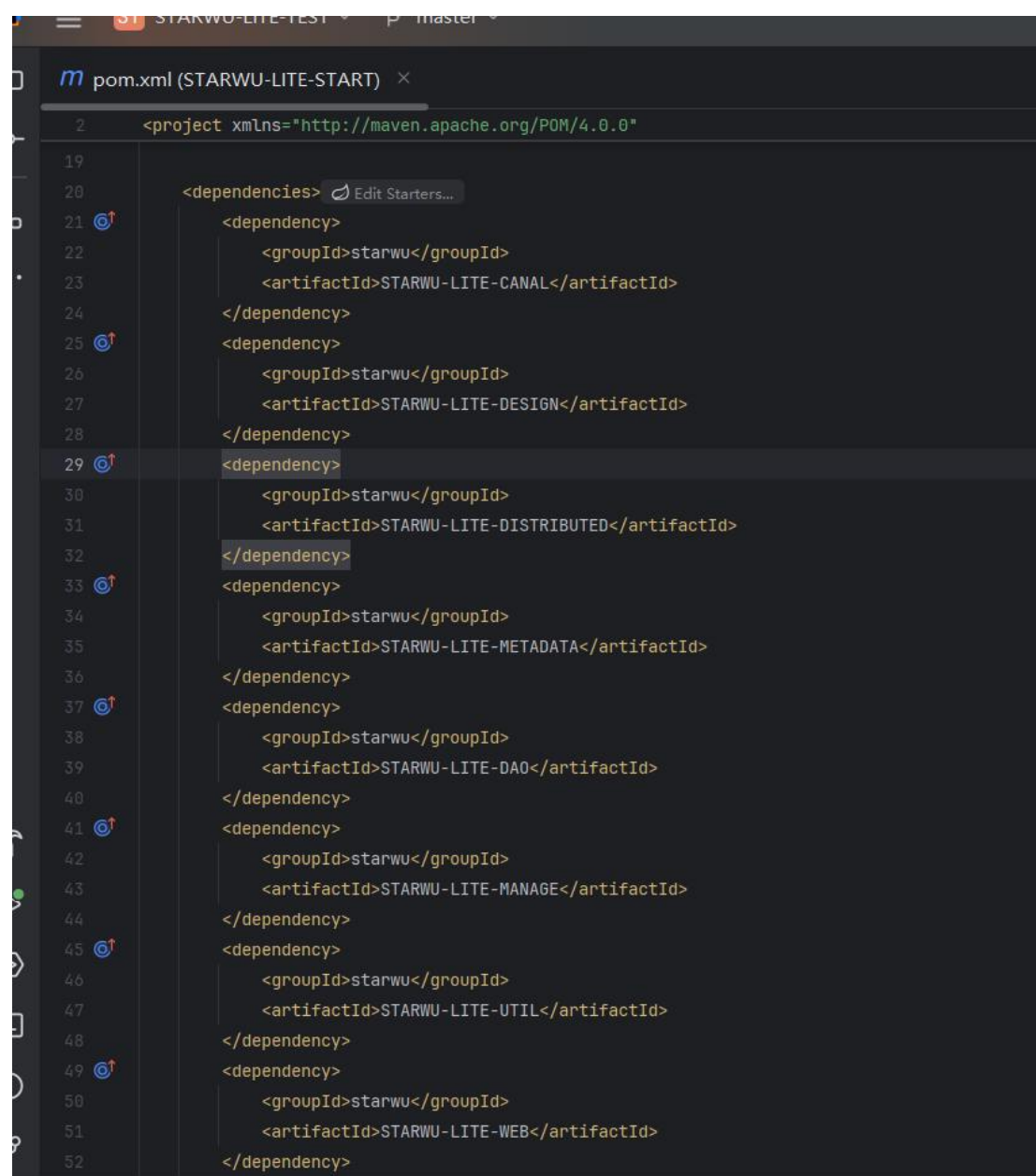


图 3-1 STARWU-LITE-START 模块依赖

在使用该框架时，直接引入 START 模块即可，如下图 3-2 所示。

```
<dependencies> Edit Starters...
  <dependency>
    <groupId>starwu</groupId>
    <artifactId>STARWU-LITE-START</artifactId>
    <version>1.0.0</version>
  </dependency>
</dependencies>
```

图 3-2 引入 STARWU-LITE-START 模块

这样引入 START 模块以后，就可以使用该框架提供的全部的功能了。

但是如果在使用的時候，我只是想使用框架提供的其中一部分功能，比如我只想使用 WEB 模块的记录请求日志的功能，其他的功能我都不用，这样其实就没有必要把所有的模块都引入进来，就只需要把 WEB 模块引入引来就可以，如下图 3-3 所示：

```
<dependencies> Edit Starters...
  <dependency>
    <groupId>starwu</groupId>
    <artifactId>STARWU-LITE-WEB</artifactId>
    <version>1.0.0</version>
  </dependency>
</dependencies>
```

图 3-3 引入 STARWU-LITE-WEB 模块

3.1.2 bean 扫描

在本框架中的除 START 模块的其他所有模块，在 resources 中都会有一个 org.springframework.boot.autoconfigure.AutoConfiguration.imports 文件，它是 SpringBoot 提供的功能，用来指定扫描框架中定义的 bean 的。在该框架中，每个 imports 文件，都会指向一个 bean，然后在该 bean 中，用 @Import 注解指定要扫描的该模块中的哪些 bean，以 WEB 模块举例，如下图 3-4 和 3-5 所示。

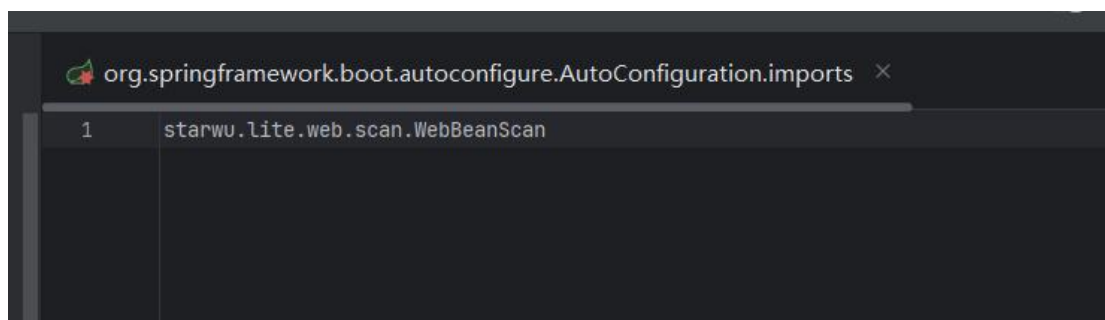


图 3-4 imports 文件

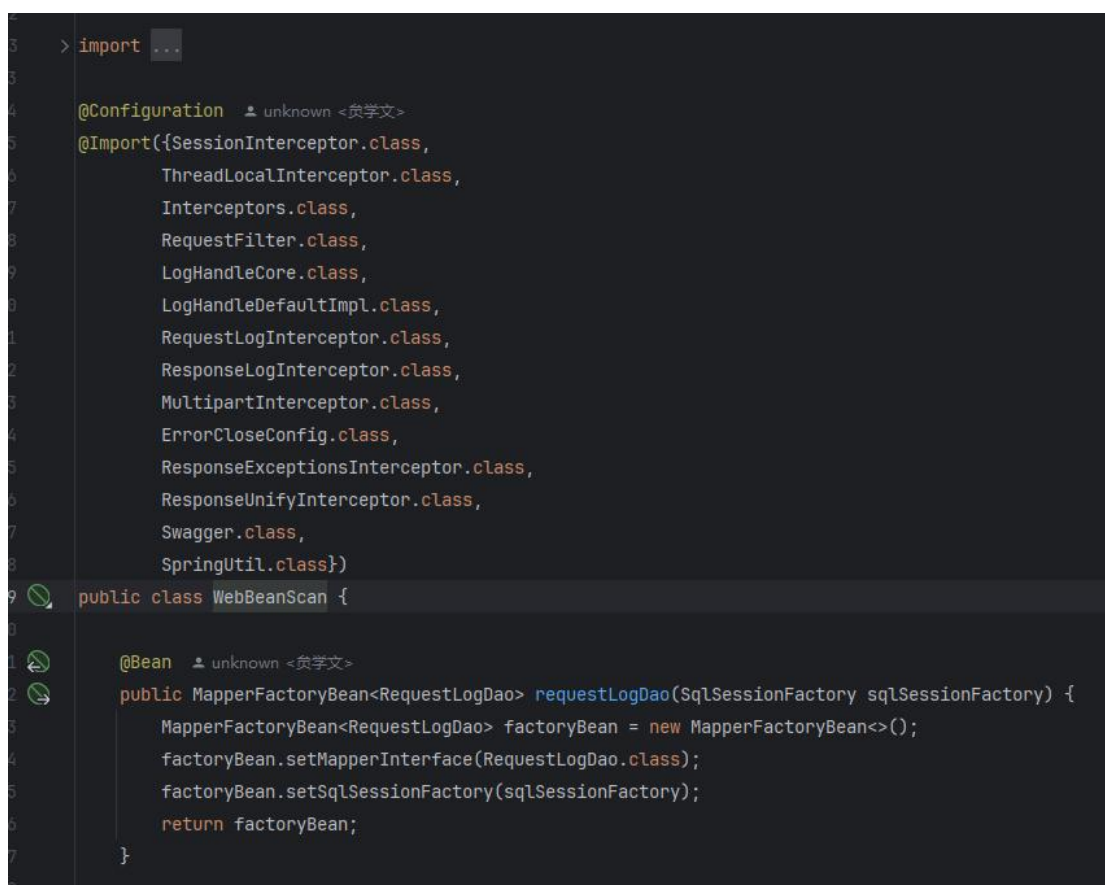


图 3-5 配置要扫描的 bean

这样，在使用该框架时，无论是直接引入 START 模块，还是引入了其他的模块，都无需考虑 bean 扫描的问题，bean 的扫描，都在框架中处理好了。

3.1.3 外部 jar 包依赖

本框架中，也依赖了一些其他的 jar 包，所有依赖的 jar 包和依赖的 jar 包版本，都在父 module 中的 pom.xml 中的<properties>中，做统一管理，如下图 3-6 所示：

```

<properties>
  <maven.compiler.source>1.8</maven.compiler.source>
  <maven.compiler.target>1.8</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
  <spring-boot.version>2.7.14</spring-boot.version>
  <fastjson2.version>2.0.62</fastjson2.version>
  <canal.version>1.1.7</canal.version>
  <mybatis-plus.version>3.5.17</mybatis-plus.version>
  <redisson.version>3.17.7</redisson.version>
  <UserAgentUtils.version>1.21</UserAgentUtils.version>
  <springfox-swagger2.version>2.9.2</springfox-swagger2.version>
  <swagger-annotations.version>1.5.20</swagger-annotations.version>
  <springfox-swagger-ui.version>2.9.2</springfox-swagger-ui.version>
  <knife4j-spring-ui.version>2.0.9</knife4j-spring-ui.version>
  <swagger-bootstrap-ui.version>1.9.4</swagger-bootstrap-ui.version>
  <starwu.lite.version>1.0.0</starwu.lite.version>
</properties>

```

图 3-6 jar 包版本管理

如果在使用该框架的过程中，觉得框架中依赖的某个 jar 包的版本太低或者太高，想去对版本做升级或者降级，可以在引入该框架时，用<exclusions>标签移除该 jar 包，然后在引入你想要的 jar 包即可。

3.2 WEB 模块

3.2.1 记录请求日志

➤ 记录请求日志的目的：

1. 某个接口出现问题时，可以根据请求日志快速定位问题。
2. 可以监控接口响应的成功率以及响应时长，从而制定出系统的优化方案。
3. 可以方便的统计用户访问某些接口的频率，从而制定出客户画像。

➤ 记录请求日志的方式：

1. 在日志文件中记录

除在日志文件中记录外，框架还支持将请求日志记录到数据库中。框架在请求进入时，由 RequestLogInterceptor 拦截器生成请求日志（包含请求唯一编号、请求

URL、请求方式、客户端 IP、客户端端口、客户端操作系统、设备类型、请求参数、线程 ID、容器 ID 等信息），并将其暂存到 ThreadLocal 中；请求处理完成后，由 ResponseLogInterceptor 记录响应参数与请求耗时，通过 LogHandleCore 以异步的方式交给 LogHandleApi 处理。框架默认实现 LogHandleDefaultImpl 会将请求与响应日志写入数据库，业务方也可以实现 LogHandleApi 接口自定义日志的存储方式。同时，框架支持通过配置 excludeRecordLogUrls 来排除不需要记录日志的 URL。

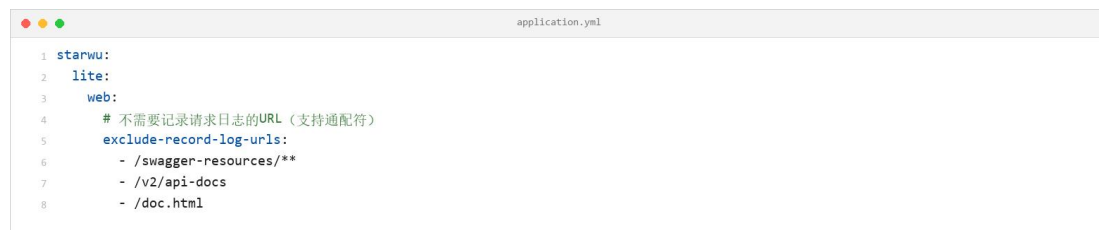


图 3-7 配置排除记录请求日志的 URL

3.2.2 响应参数封装

在前后端分离的开发模式下，如果每个接口各自定义返回格式，前端就需要针对不同的接口做不同的解析，维护成本很高。因此，框架对所有接口的返回参数做了统一封装。

框架通过 ResponseUnifyInterceptor 实现响应参数的统一封装。该类基于 SpringMVC 的 @RestControllerAdvice 与 ResponseBodyAdvice 接口实现，在接口返回数据之前，将 Controller 的返回值统一封装为 ResponseBean 对象。ResponseBean 包含四个字段：result（返回结果，取值为枚举 ResponseResult 中的 SUCCESS 或 FAIL）、errCode（错误码，取值为枚举 ErrorCodeType）、errMsg（返回信息）、data（返回数据）。

如果 Controller 的返回类型是 String，由于 SpringMVC 会使用 StringHttpMessageConverter 转换器进行处理，直接封装会抛出 ClassCastException，因此框架对这种类型做了特殊处理：先将返回数据封装为 ResponseBean，再序列化为 JSON 字符串返回，同时将响应的 ContentType 设置为 APPLICATION_JSON，保证前端可以统一按 JSON 格式解析。

对于不需要统一封装的接口（比如第三方回调接口），框架支持通过配置 excludeUnifyRespUrls 排除指定的 URL，被排除的 URL 将原样返回。

```

1 @RestController
2 @RequestMapping("/user")
3 public class UserController {
4
5     @Autowired
6     private UserService userService;
7
8     // 直接返回业务数据即可，框架会自动封装为统一的 ResponseBean
9     @GetMapping("/getById")
10    public User getById(Long id) {
11        return userService.getById(id);
12    }
13 }

```

图 3-8 响应参数封装使用示例

3.2.3 统一异常处理

在一个项目中，如果每个接口各自处理异常，会导致大量的 try-catch 重复代码，而且返回格式难以统一。框架通过 ResponseExceptionsInterceptor 实现统一的异常处理，该类基于 @RestControllerAdvice 注解实现，针对不同异常分别定义了 @ExceptionHandler 处理方法，异常发生时统一封装为 ResponseBean 返回给前端。框架处理的异常及对应的错误码如下：

- (1) SessionTimeoutException: Session 过期异常，错误码 1005;
- (2) BindException、ParamValidationException: 参数校验异常，错误码 1003;
- (3) PermissionValidationException: 权限校验异常，错误码 1004;
- (4) SQLException: 数据库异常，对外统一提示“系统异常，请稍后重试”，错误码 1001;
- (5) NoHandlerFoundException: 404 异常，即找不到对应的接口，错误码 1006;
- (6) BusinessException: 业务异常，直接返回异常中携带的业务提示信息，错误码 1002;
- (7) Throwable: 所有未预期异常的兜底处理，对外统一提示“系统异常，请稍后重试”，错误码 1001。

所有异常在被捕获时，都会打印 error 级别的堆栈日志，方便开发人员排查问题。

```

1 @Service
2 public class UserService {
3
4     public void recharge(Long userId, BigDecimal amount) {
5         if (amount.compareTo(BigDecimal.ZERO) <= 0) {
6             // 抛出业务异常，框架统一封装为错误码1002返回给前端
7             throw new BusinessException("充值金额必须大于0");
8         }
9         // 执行业务逻辑...
10    }
11 }

```

图 3-9 统一异常处理使用示例

3.2.4 文件上传

文件上传场景中，经常出现“文件+JSON 数据”混合提交的 multipart/form-data 请求，请求中除文件外还包含 application/octet-stream 类型的数据，SpringMVC 默认没有合适的 HttpMessageConverter 处理这类数据，导致数据无法解析。

框架通过 MultipartInterceptor 解决该问题。MultipartInterceptor 继承 AbstractJackson2HttpMessageConverter，注册 application/octet-stream 媒体类型的转换能力，并屏蔽其写能力（canWrite 返回 false），使其只参与请求的反序列化；在 Interceptors 配置类中通过 extendMessageConverters 将其注册到 SpringMVC 的转换器列表中。这样，业务接口就可以在一个请求中同时接收文件和二进制流形式的数据对象，框架会自动将其反序列化为对应的 Java 对象。



```
1 @RestController
2 public class FileController {
3
4     // 一个请求同时接收文件和数据对象，框架自动完成反序列化
5     @PostMapping("/upload")
6     public String upload(@RequestPart("file") MultipartFile file,
7                         @RequestPart("user") User user) {
8
9         // 处理文件与数据对象...
10        return file.getOriginalFilename();
11    }
12 }
```

图 3-10 文件上传使用示例

3.2.5 swagger

框架集成了 Swagger 用于自动生成接口文档，基于 springfox-swagger2 实现，并集成了 swagger-bootstrap-ui，提供 doc.html 页面的文档展示效果。

Swagger 配置类通过 @ConditionalOnClass 注解实现条件装配，只有在项目中存在 Swagger 相关类时才会生效。框架默认扫描所有标注了 @Api 注解的 Controller 生成接口文档；文档的标题、描述、版本、作者、是否开启等信息均可在配置文件中自定义；框架支持配置鉴权请求头（authHeaders），并在文档页面中自动生成鉴权参数与全局 token 参数，方便在文档页面直接调试需要登录的接口。

项目启动完成后，框架会自动打印 Swagger 接口文档的访问地址（http://IP:端口/doc.html），方便开发人员查看。此外，框架通过 BeanPostProcessor 对 springfox 的 handlerMappings 做了兼容处理，解决了较高版本 SpringBoot 与 Swagger2 的兼容问题。



图 3-11 Swagger 配置示例

3.3 DAO 模块

3.3.1 数据库与 redis 的结合使用

框架基于 MyBatis-Plus 对数据库操作做了进一步封装，DAO 模块的核心入口是 BaseDao 接口，它继承 MyBatis-Plus 的 BaseMapper，业务方的 DAO 接口只需继承 BaseDao 即可获得数据库与 Redis 结合使用的能力。

具体使用方式如下：业务实体类继承 BaseEntity，并在实体类上标注框架提供的 @Cache 注解，即表示该实体的数据需要缓存，注解的 period 属性用于指定缓存有效期（不指定时使用全局配置）。调用 BaseDao 的 getById 方法查询数据时，框架会先根据实体元数据生成 RedisKey 查询 Redis：如果命中缓存则直接返回；如果命中空值标记则直接返回 null；如果未命中，则根据配置决定直接查询数据库回写缓存，还是通过分布式锁查询数据库（详见 3.3.2 节）。

业务方还可以实现 RedisCooperationApi 接口，自定义 RedisKey 的生成规则、查询数据库的逻辑、结果的序列化方式、缓存超时时间以及加锁重试策略，从而完全掌控数据库与 Redis 的交互过程。



图 3-12 数据库与 Redis 结合使用示例

3.3.2 redis 的缓存击穿

缓存击穿是指某个热点数据在缓存过期的瞬间，大量并发请求同时打到数据库，导致数据库压力骤增甚至崩溃。

框架基于 Redisson 分布式锁解决缓存击穿问题。当缓存未命中且配置需要加锁

(isNeedTryLock 返回 true) 时，框架以 RedisKey 加“-LOCK”后缀作为锁的 Key，尝试获取分布式锁：获取锁成功的线程负责查询数据库并将结果回写缓存（包括空值标记），然后释放锁；获取锁失败的线程则休眠指定时间（getSleepTime）后重试，重新从缓存中读取；如果重试次数超过配置的最大次数（getReTryCount）仍未获取到数据，则返回约定的兜底结果（getTryLockFailResult）。通过这种方式，保证同一时间只有一个线程访问数据库，其余线程均从缓存中获取数据。

```
UserRedisApi.java
1 @Component
2 public class UserRedisApi implements RedisCooperationApi<Long, User> {
3
4     @Autowired
5     private UserDao userDao;
6
7     // 开启分布式锁，防止热点数据过期时的缓存击穿（3.3.2节）
8     @Override
9     public boolean isNeedTryLock() { return true; }
10
11     @Override
12     public int getReTryCount() { return 6; } // 加锁重试次数
13
14     @Override
15     public long getSleepTime() { return 50; } // 重试间隔(毫秒)
16
17     @Override
18     public User getFromDatabase(Long id) {
19         // 查库结果为空时，框架自动写入空值标记，防止缓存穿透（3.3.3节）
20         return userDao.selectById(id);
21     }
22
23     @Override
24     public String getRedisKey(Long id) { return "USER:" + id; }
25
26     @Override
27     public int redisTimeOut() { return 30; } // 缓存有效期(分钟)
28
29     // convertToResult、getTryLockFailResult 等方法同理实现
30 }
```

图 3-13 防止缓存击穿的加锁配置

3.3.3 redis 的缓存穿透

缓存穿透是指查询一个数据库中根本不存在的数据库，导致每次请求都绕过缓存直接打到数据库，如果有人恶意利用这类请求，会对数据库造成很大压力。

框架采用缓存空值的方式解决缓存穿透问题。当查询数据库的结果为空时，框架不会直接返回，而是向 Redis 中写入一个空字符串作为标记，并设置过期时间。后续的相关请求命中该空值标记后，直接返回 null，不再访问数据库，从而保护了数据库。

```
UserService.java
1 // 缓存穿透由框架自动处理，业务零配置；
2 // 查询数据库结果为空时，框架自动向Redis写入空值标记并设置过期时间，
3 // 后续相同请求命中空值标记后直接返回null，不再访问数据库。
4 User user = userDao.getById(); // 该用户不存在
5
6 // 第一次查询：访问数据库，返回null，同时写入空值标记
7 // 第二次查询：直接命中空值标记返回null，不再访问数据库
```

图 3-14 缓存穿透的自动处理

3.3.4 redis 与数据库的数据一致性

在使用 Redis 缓存后，一个典型的问题是：数据库中的数据被修改后，如何及时同步到 Redis，保证两者数据一致。框架通过集成 Canal 监听数据库的 binlog 日志来解决该问题（Canal 的监听原理详见 3.4 节）。

DAO 模块中的 CanalRedisSubscribeCore 作为数据库变更的订阅者，接收到变更数据后，通过 CanalRedisFactory 工厂获取对应的处理器 CanalRedisHandleApi，默认实现为 CanalRedisDefaultHandleImpl，其处理逻辑如下：

- (1) 初始化时，通过反射读取 MyBatis-Plus 中实体类与数据库表的映射关系，建立“表名-实体元数据”的映射缓存，并采用双重检查锁保证只初始化一次；
- (2) 只有标注了 @Cache 注解的实体对应的表发生变更时才需要处理，其余表的变更直接忽略；
- (3) 新增（INSERT）、修改（UPDATE）时，如果是热点数据则直接更新 Redis 缓存，否则删除对应的缓存；
- (4) 删除（DELETE）时，直接删除对应的缓存。

框架还提供了 isHotSpotData、insertOtherHandle、updateOtherHandle、deleteOtherHandle 等扩展点，业务方可以重写默认实现，自定义热点数据的判断逻辑以及变更后的其他处理逻辑。通过“先更新数据库，再依赖 Canal 同步缓存”的方式，避免了业务代码中手动维护缓存的繁琐与不一致风险。



```
1 // 继承默认实现并重写扩展点，启动后自动生效
2 @Component
3 public class UserRedisHandle extends CanalRedisDefaultHandleImpl {
4
5     // 重写热点数据判断逻辑：热点数据变更时直接更新缓存，否则删除缓存
6     @Override
7     protected boolean isHotSpotData(JSONObject afterJson) {
8         return afterJson.getInteger("level") >= 3;
9     }
10
11     // 数据变更后的其他自定义处理
12     @Override
13     protected void updateOtherHandle(JSONObject afterJson) {
14         // 例如：同步刷新相关的统计缓存
15     }
16 }
```

图 3-15 数据一致性处理的扩展示例

3.3.5 记录数据库的变更日志

在很多业务场景中，需要审计数据库中的数据是被谁在什么时候改成了什么样，框架基于 Canal 提供了数据库变更日志的记录能力。

DAO 模块中的 CanalSubscribeSubscribeCore 作为数据库变更的订阅者，接收到变更

数据后，交由 CanalDblogHandleApi 处理，默认实现 CanalDblogHandleDefaultImpl 会将每次变更记录写入数据库的变更日志表，包括：数据库名称、表名称、变更执行时间、变更前的数据、变更后的数据、主键值以及记录创建时间。其中，新增和修改操作取变更后数据的主键，删除操作取变更前数据的主键。

业务方可以实现 CanalDblogHandleApi 接口并通过 CanalDblogFactory 注册，来自定义变更日志的记录方式，比如记录到日志文件或其他存储介质。



```
1 // 默认实现会自动将数据库变更写入变更日志表，业务零配置。
2 // 如需自定义记录方式，实现 CanalDblogHandleApi 接口即可。
3 // 框架启动时自动注册并优先生效：
4 @Component
5 public class MyDblogHandle implements CanalDblogHandleApi {
6
7     @Override
8     public void handle(CanalDataBean canalDataBean) {
9         // 自定义处理：例如记录到日志文件或其他存储介质
10        log.info("数据库变更: {} 主键: {}",
11                canalDataBean.getTableName(),
12                canalDataBean.getAfer().getLong("id"));
13    }
14 }
```

图 3-16 自定义数据库变更日志记录

3.3.6 监控慢 sql

随着业务数据量的增长，慢 SQL 会严重影响系统性能，因此需要及时发现并优化。框架基于 MyBatis 拦截器机制实现了慢 SQL 的监控。

DaoSqlInterceptor 拦截了 MyBatis 执行 SQL 的 query、queryCursor、update、batch 四类方法，在 SQL 执行前后记录时间戳，计算 SQL 的实际执行耗时，如果耗时超过配置文件中设置的阈值（sqlExecTimeOut），则认定为慢 SQL，交由 SlowSqlHnadleCore 处理。

为避免记录慢 SQL 的过程影响主业务流程，框架通过名为 recordSlowSql 的独立异步线程池异步处理。记录的慢 SQL 信息包括：SQL 语句、开始时间、结束时间、耗时、线程 ID、容器 ID，以及当前请求的信息（请求唯一编号）和当前登录用户（用户 ID、用户名），方便定位慢 SQL 是由哪个用户的哪个请求触发的。同时，为避免框架自身的慢 SQL 日志写入语句被重复记录，框架对其做了排除处理。

框架默认实现 SlowSqlHandleDeaultImpl 会将慢 SQL 信息打印到日志文件并写入数据库的慢 SQL 日志表，业务方可以实现 SlowSqlHandleApi 接口自定义处理方式。



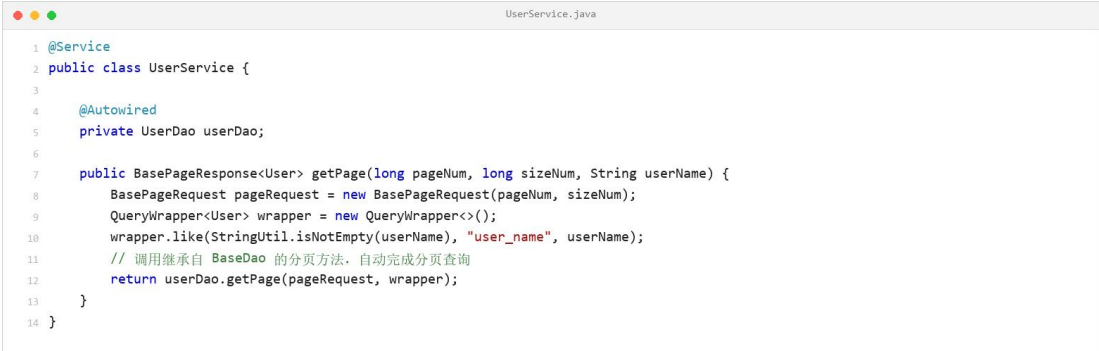
```
1 starwu:
2   lite:
3     dao:
4       sql-exec-time-out: 1000 # SQL执行耗时阈值(毫秒)，超过则记录慢SQL
5       cache-period: 30      # 缓存默认有效期(分钟)
```

图 3-17 慢 SQL 监控配置

3.3.7 mybatisPlus 的分页

分页是后台管理类系统中最常用的功能之一。框架通过 DaoPageQueryInterceptor 向 MyBatis-Plus 注册了分页插件 PaginationInnerInterceptor（适配 MySQL 数据库），业务方无需任何配置即可使用分页能力。

在代码层面，框架提供了统一的分页请求对象 BasePageRequest（包含页码 pageNum 和每页条数 sizeNum）和分页响应对象 BasePageResponse（包含总条数、总页数和当前页数据列表）。业务方的 DAO 接口继承 BaseDao 后，调用其 getPage 方法并传入分页请求对象和查询条件构造器，即可完成分页查询，框架会自动组装分页结果返回。



```
1 @Service
2 public class UserService {
3
4     @Autowired
5     private UserDao userDao;
6
7     public BasePageResponse<User> getPage(long pageNum, long sizeNum, String userName) {
8         BasePageRequest pageRequest = new BasePageRequest(pageNum, sizeNum);
9         QueryWrapper<User> wrapper = new QueryWrapper<>();
10        wrapper.like(StringUtils.isNotEmpty(userName), "user_name", userName);
11        // 调用继承自 BaseDao 的分页方法，自动完成分页查询
12        return userDao.getPage(pageRequest, wrapper);
13    }
14 }
```

图 3-18 分页查询使用示例

3.4 CANAL 模块

3.4.1 监听数据库的变更

Canal 是阿里巴巴开源的数据库 binlog 日志解析组件，可以把它理解为一个“数据库的监听器”：它伪装成 MySQL 的从库，实时接收 MySQL 主库推送的 binlog 日志，从而感知数据库的所有数据变更。

框架的 CANAL 模块对 Canal 客户端做了封装，核心类是 TcpListener。项目启动时，TcpListener 通过 @PostConstruct 注解自动启动监听线程，根据配置（Canal 服务端地址，支持集群多地址、destination、用户名密码、订阅过滤表达式、批量拉取条数、超时时间等）创建 CanalConnector 连接 Canal 服务端，并订阅感兴趣的库表。监听线程循环拉取 Canal 推送的消息，对每条消息进行确认（ack），并将消息解析为框架统一的 CanalDataBean 对象，包含数据库名称、表名称、变更执行时间、变更类型（INSERT/UPDATE/DELETE）以及变更前、变更后的数据，之后交由 CanalSubject 推送给所有订阅者。

整个解析与推送过程在框架配置的异步线程池中执行，不阻塞监听线程；处理过程中的异常会被捕获并记录日志，不影响后续消息的接收。



图 3-19 Canal 监听配置

3.4.2 将数据库的变更推送给订阅者

数据库的变更可以被用于多种场景：同步 Redis 缓存、记录变更日志、事务补偿等。为了解耦“变更的监听”与“变更的使用”，框架基于观察者模式实现了变更数据的推送机制。

框架定义了被观察者抽象类 SubjectApi 与观察者接口 CanalObserverApi。订阅者只需实现 CanalObserverApi 接口（isReceived 方法判断是否接收该条变更，received 方法处理变更），并在启动时注册到 CanalSubject 中，即可接收到数据库的变更数据。CanalSubject 接收到变更后，会通过异步线程池并行通知所有订阅者，每个订阅者在独立的线程中处理，互不影响。

框架内部基于该机制实现了多个订阅者：DAO 模块的 CanalRedisSubscribeCore（解决缓存一致性问题）、CanalSubscribeSubscribeCore（记录数据库变更日志），DISTRIBUTED 模块的 CanalTransactionSubscribeCore（分布式事务补偿，详见 3.5.2 节）。业务方也可以很方便地扩展自己的订阅者。



图 3-20 订阅数据库变更示例

3.5 DISTRIBUTED 模块

3.5.1 分布式 session

在分布式部署环境下，同一个用户的多次请求可能被负载均衡到不同的服务器，传统的基于 Tomcat 的 Session 无法共享，导致用户登录状态丢失。框架基于 Redis 实现了分布式 Session。

SessionInterceptor 拦截器拦截所有请求（可通过配置 `excludeUrls` 排除登录等接口），从请求头中获取 token，交由 SessionCore 校验：如果 token 为空或 Redis 中不存在对应的 Session，则抛出 `SessionTimeOutException`，由 WEB 模块统一异常处理返回“请重新登录”的提示；校验通过后会重置 Session 的有效期（滑动过期），并将 Session 信息存入 ThreadLocal，方便后续业务代码获取当前登录用户。

Session 在 Redis 中以两种 Key 存储：以 token 为 Key 存储完整的 Session 对象（用户 ID、用户名、设备类型及业务自定义数据），以“用户 ID+设备类型”为 Key 存储 token 用于登录互踢；此外，框架还通过 Redis 的有序集合维护在线用户，可统计用户在线人数。

登录时，业务方调用 SessionCore 的 `createSession` 方法，框架通过雪花算法生成全局唯一的 token（TOKEN- 前缀），写入 Redis 并通过响应头返回给前端。框架支持三种登录互踢策略（可配置）：不踢出、同一用户只允许一个登录（踢出旧登录）、同一用户同一设备只允许一个登录。

该方案使 Session 完全与具体服务器解耦，无论请求被转发到哪台服务器，都能获取到统一的登录状态。



```
1 @RestController
2 public class LoginController {
3
4     @Autowired
5     private SessionCore sessionCore;
6
7     @Autowired
8     private UserService userService;
9
10    @PostMapping("/login")
11    public String login(String userName, String password,
12                        HttpServletResponse response) {
13        User user = userService.checkLogin(userName, password);
14        // 创建分布式Session, token自动写入响应头返回给前端,
15        // 之后的请求在请求头中携带token即可, 支持配置登录互踢策略
16        return sessionCore.createSession(user.getId(), 1,
17                                         user.getUserName(), user, response);
18    }
19 }
```

图 3-21 分布式 Session 使用示例

3.5.2 分布式事务

在分布式系统中，一个业务操作往往需要调用多个服务的接口，如何保证这些操作要

么全部成功、要么全部失败，是分布式事务要解决的问题。框架采用“本地消息表 + Canal 实时驱动 + 定时补偿”的方案实现了分布式事务（最终一致性）。核心流程如下：

(1) 业务方在本地事务中调用 TransactionCore 的 add 方法，将待执行的远程调用记录（目标 URL、调用参数、来源请求信息、当前用户等）写入本地的事务表，与业务数据在同一个本地事务中提交，保证业务数据与调用记录的原子性；

(2) 写入事务表后，CanalTransactionSubscribeCore 订阅到事务表的 INSERT/UPDATE 变更，根据目标 URL 通过 TransactionRemoteCallFactory 工厂找到业务方实现的 TransactionRemoteCallApi 接口，立即发起远程调用：调用成功执行 successHandle，更新事务状态并回调 afterSuccessHandle；调用失败执行 failHandle，记录失败信息到事务日志表，如果未达到重试上限则标记为待定时重试；

(3) ScheduledTask 定时任务每 5 分钟扫描一次标记为待重试的事务，重新发起调用，直到成功或达到重试上限（默认 3 次，可通过 retryCount 自定义）。

通过上述机制，即使服务间调用失败或系统宕机，未完成的分布式操作也会被可靠地补偿执行，保证数据的最终一致性。业务方只需实现 TransactionRemoteCallApi 接口（call 方法发起调用，可选重写 afterSuccessHandle、afterFailHandle、retryCount），并注册到 TransactionRemoteCallFactory 即可使用。



```
1 // 1.本地事务中记录待执行的远程调用，与业务数据在同一个事务中提交
2 @Transactional(rollbackFor = Exception.class)
3 public void createOrder(OrderParam param) {
4     orderDao.insert(param); // 业务数据
5     // 记录对订单服务的远程调用，框架自动驱动调用，失败自动补偿重试
6     transactionCore.add("orderRemoteCall", param);
7 }
8
9 // 2.实现 TransactionRemoteCallApi 接口，定义对目标服务的调用，
10 // 并按目标URL注册到 TransactionRemoteCallFactory
11 @Component
12 public class OrderRemoteCall implements TransactionRemoteCallApi {
13     @Override
14     public ResponseBean call(JSONObject json) {
15         return HttpUtil.post("http://order-service/create", json);
16     }
17 }
```

图 3-22 分布式事务使用示例

3.6 DESIGN 模块

3.6.1 工厂模式

工厂模式用于根据不同的条件创建不同的处理对象。框架将工厂模式封装为抽象类 FactoryEntryApi 与处理器接口 FactoryHandleApi，业务方只需继承这两个类即可获得完整的工厂能力：

(1) 业务处理器实现 FactoryHandleApi 接口，重写 match 方法定义自己的匹配规则；

(2) 业务工厂继承 FactoryEntryApi，启动时将各处理器通过 regist 方法注册到工厂，并实现 getDefaultCls 方法指定默认处理器；

(3) 使用时调用工厂的 get 方法并传入匹配 Key，工厂先查询缓存，未命中时遍历已注册的处理器，将 Key 交给各处理器的 match 方法匹配，匹配成功则返回该处理器并缓存匹配结果，全部不匹配时返回默认处理器。

框架内部广泛使用该封装：DAO 模块的 CanalRedisFactory、CanalDblogFactory，DISTRIBUTED 模块的 TransactionRemoteCallFactory 等均基于它实现，业务方也可以基于它快速实现自己的工厂。



```
PayHandle.java
1 // 1.定义处理器：重写 match 方法定义匹配规则，
2 // 框架启动时自动注册到 getFactoryCls 指定的工厂
3 @Component
4 public class AliPayHandle implements FactoryHandleApi<String> {
5
6     @Override
7     public boolean match(String key) {
8         return "aliPay".equals(key);
9     }
10
11     @Override
12     public Class<? extends FactoryEntryApi> getFactoryCls() {
13         return PayFactory.class;
14     }
15 }
16
17 // 2.定义工厂：指定默认处理器
18 @Component
19 public class PayFactory extends FactoryEntryApi<String, FactoryHandleApi> {
20     @Override
21     public Class getDefDefaultCls() { return DefaultPayHandle.class; }
22 }
23
24 // 3.使用：传入Key自动获取对应的处理器，无匹配时返回默认处理器
25 FactoryHandleApi handle = payFactory.get("aliPay");
```

图 3-23 工厂模式使用示例

3.6.2 责任链模式

责任链模式用于将多个处理器按顺序串成一条链，依次处理同一个请求。框架将其封装为抽象类 ChainEntryApi 与处理器接口 ChainHandleApi：

(1) 业务处理器实现 ChainHandleApi 接口，重写 order 方法指定处理顺序（数值越小越先执行），重写 isMatch 方法判断是否处理该请求，重写 process 方法执行具体处理逻辑并返回布尔值；

(2) 业务责任链继承 ChainEntryApi，通过 regist 方法注册处理器，注册时框架会按照 order 值自动完成排序；

(3) 处理请求时调用 process 方法，框架按顺序依次执行每个匹配处理器的 process

方法，一旦某个处理器返回 false，责任链立即终止，后续处理器不再执行。

通过该封装，业务方可以像搭积木一样组合多个处理器，灵活控制请求的处理流程。

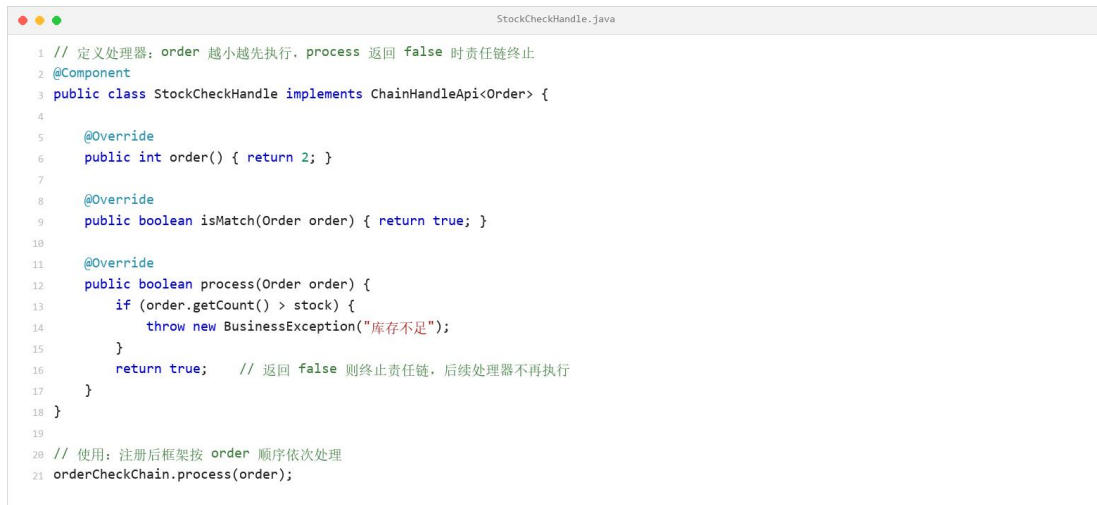


图 3-24 责任链模式使用示例

3.6.3 观察者模式

观察者模式用于实现一对多的消息通知：当被观察的对象发生变化时，所有观察者都会收到通知。框架将其封装为抽象类 SubjectApi（被观察者）与接口 ObserverApi（观察者）：

（1）观察者实现 ObserverApi 接口，重写 isReceived 方法决定是否接收消息，重写 received 方法处理消息；

（2）被观察者继承 SubjectApi，通过 regist 方法注册观察者；

（3）通知时调用 notify 方法，框架会使用配置的异步线程池，在独立线程中并行通知所有观察者，某个观察者处理失败不会影响其他观察者。

CANAL 模块的 CanalSubject 即基于该封装实现（详见 3.4.2 节），业务方也可以基于它实现任何事件驱动的通知机制。



```

1 // 定义观察者：指定所属被观察者，框架启动时自动注册
2 @Component
3 public class SmsObserver implements ObserverApi<Order> {
4
5     @Override
6     public Class<? extends SubjectApi> getSubjectCls() {
7         return OrderSubject.class;
8     }
9
10    @Override
11    public boolean isReceived(Order order) { return order != null; }
12
13    @Override
14    public void received(Order order) {
15        // 异步执行：发送下单短信通知
16        smsService.send(order);
17    }
18 }
19
20 // 使用：发布消息，框架使用异步线程池通知所有观察者
21 orderSubject.notify(order);

```

图 3-25 观察者模式使用示例

3.7 MANAGE 模块

3.7.1 ThreadLocal 功能增强

ThreadLocal 是 Java 中实现线程内数据隔离的常用工具，但在实际使用中经常遇到两个问题：一是使用完忘记清理，在线程池复用线程的场景下会导致数据串扰和内存泄漏；二是缺乏统一的 Key 管理，容易相互冲突。

框架通过 ThreadLocalCore 对 ThreadLocal 做了增强封装：内部维护两个 ThreadLocal（Map 结构），一个用于存放请求结束后需要清除的数据（如 Session 信息、请求日志），一个用于存放跨请求常驻的数据（通过 isNeedRemove 参数控制）；提供 get、put、clear 等静态方法，并通过 ThreadLocalKey 常量类统一管理 Key；WEB 模块的 ThreadLocalInterceptor 拦截所有请求，请求处理完成后自动调用 clear 方法清理 ThreadLocal，杜绝内存泄漏与线程复用导致的数据串扰。

框架内部大量使用 ThreadLocalCore 传递请求上下文：记录请求日志时暂存 RequestLog，Session 校验后暂存 UserSession，分布式事务记录来源请求信息等。业务方也可以通过它安全地在线程内传递数据。



```

1 // 在任意位置安全地存取线程内数据，请求结束后框架自动清理
2 ThreadLocalCore.put("currentShopId", shopId);
3 Long shopId = ThreadLocalCore.get("currentShopId");
4
5 // 跨请求常驻的数据可放入不自动清理的ThreadLocal
6 ThreadLocalCore.put("configCache", config, false);
7
8 // 请求处理完成后，ThreadLocalInterceptor 自动调用 clear 清理。
9 // 无需业务代码手动处理

```

图 3-26 ThreadLocal 增强使用示例

3.7.2 Spring 异步功能增强

Spring 的 @Async 异步功能默认只有一个简单的线程池配置，多个异步场景共用时容易相互影响。框架对异步线程池做了增强封装，由 AsyncThreadPool 统一管理：

(1) 支持在配置文件中定义多个命名线程池，每个线程池可独立配置核心线程数、最大线程数、队列容量、线程空闲存活时间、线程名前缀、拒绝策略（枚举 ThreadPoolRejectedExecutionHandlerEnum 提供 JDK 的全部四种拒绝策略）、关闭时是否等待任务执行完成；

(2) 项目启动时，框架根据配置动态创建 ThreadPoolTaskExecutor 并注册为 Spring 单例 Bean，同时提供一个 default 默认线程池；框架初始化完成后，业务方仍可通过 addThreadPoolTaskExecutor 方法动态添加新的线程池；

(3) AsyncExceptionHandler 对异步任务未捕获的异常做统一处理。

框架内部的 Canal 消息处理、观察者通知、请求日志记录、慢 SQL 记录等异步场景，均使用该组件创建的线程池，实现了异步资源的全局统一管理。



图 3-27 异步线程池配置示例

3.8 UTIL 模块

UTIL 模块定义了框架内部及各业务项目中常用的工具类，目前包含以下内容：

(1) 字符串与日期：StringUtil（字符串拼接等常用操作）、DateUtil（日期计算与格式化）；

(2) 集合与文件：CollectionUtil（集合判空与转换）、FileUtil（文件操作）、PathUtil（路径处理）；

(3) 系统与环境：OSUtil（操作系统判断）、HostUtil（主机信息获取）、EnvironmentUtil（容器 ID 获取，用作雪花算法的机器标识）；

(4) 反射与类操作：ClassUtil、ClassLoaderUtil、ReflectUtil（反射读写字段与方法）、GenericityUtil（泛型解析）、PackageUtil（包扫描）；

(5) 序列化与脱敏：SerializeUtil（对象序列化）、DesensitizeUtil（敏感数据脱敏）；

(6) Spring 相关：SpringUtil（在非 Spring 管理的类中获取 Spring Bean，框架内部大量使用）；

(7) ID 生成：SnowflakeIdGenerator（雪花算法全局唯一 ID 生成，采用 10bit 机器 ID 与 12bit 序列号，并对时钟回拨做了兼容处理：小幅度回拨自动等待时钟追上，大幅度回拨抛出异常）；

(8) HTTP 请求：HttpUtil，基于 HttpApi 接口与 HttpDefaultImpl 默认实现封装，业务方可以实现 HttpApi 接口自定义 HTTP 请求的行为。



图 3-28 常用工具类使用示例

3.9 METADATA 模块

METADATA 模块定义了其他各模块共用的元数据信息，本身不包含业务逻辑，是框架的“公共底座”，主要包含以下内容：

(1) 注解：如 @Cache（标记实体类需要缓存）；

(2) 枚举：如 ResponseResult（响应结果）、ErrorCodeType（错误码）、DbUpdateType（数据库变更类型）、LoginKickOutType（登录互踢策略）、CallTimeType（事务调用时机）、BooleanType（布尔枚举）、ThreadPoolRejectedExecutionHandlerEnum（线程池拒绝策略）等；同时提供 EnumApi 接口与 EnumPlus 工具类，实现枚举的统一管理（按 ID 获取枚举、序列化支持等）；

(3) 配置类：如 WebConfig（WEB 模块配置）、SwaggerConfig（Swagger 配置）、DaoConfig（DAO 模块配置）、CanalConfig（Canal 配置）、SessionConfig（分布式 Session 配置）、AsyncConfig（异步线程池配置）等，业务方均可在配置文件中按需覆盖；

(4) 实体与通用 Bean：如 BaseEntity（业务实体基类）、BaseLogEntity（日志实体

基类)、RequestLog (请求日志)、SlowSqlLog (慢 SQL 日志)、DbUpdateLog (数据库变更日志)、Transaction/TransactionLog (分布式事务)、ResponseBean (统一响应)、BasePageRequest/BasePageResponse (分页)、UserSession (用户会话)、CanalDataBean (数据库变更数据)、EntityMetadata (实体元数据) ;

(5) 异常: BusinessException (业务异常)、ParamValidationException (参数校验异常)、PermissionValidationException (权限校验异常)、SessionTimeoutException (Session 过期异常)、SQLException (数据库异常) 。



图 3-29 覆盖框架默认配置示例

4 核心类与接口说明

模块	类 / 接口名称	所在包路径	核心功能简述
WEB 模块	RequestLogInterceptor	starwu.lite.web.log.interceptor	在日志文件中记录请求日志
WEB 模块	ResponseLogInterceptor	starwu.lite.web.log.interceptor	在日志文件中记录响应日志
WEB 模块	LogHandleDefaultImpl	starwu.lite.web.log.handler	在数据库中记录请求与响应日志
WEB 模块	LogHandleApi	starwu.lite.web.log.handler	通过实现此接口，来实现自定义记录请求日志
WEB 模块	ResponseExceptionHandler	starwu.lite.web.response	统一异常处理
WEB 模块	ResponseUnifyInterceptor	starwu.lite.web.response	响应参数的统一封装
WEB 模块	MultipartInterceptor	starwu.lite.web.multipart	实现文件上传
WEB 模块	Swagger	starwu.lite.web.swagger	集成 swagger
DAO 模块	BaseDao	starwu.lite.dao.core	数据库与 redis 的结合使用
DAO 模块	CanalRedisDefaultHandleImpl	starwu.lite.dao.redis.dataConsistency.impl	解决数据库与 redis 的数据一致性
DAO 模块	CanalDblogHandlerDefaultImpl	starwu.lite.dao.dblog.impl	记录数据库数据的变更日志
DAO 模块	DaoSqlInterceptor	starwu.lite.dao.interceptor.sql	监控数据库的慢sql
CANAL 模块	TcpListener	starwu.lite.canal.listener	监听 canal 的数据库的变更信息
CANAL 模块	CanalSubject	starwu.lite.canal.notify	将监听到的数据库变更信息通知到订阅者
DISTRIBUTED 模块	SessionCore	starwu.lite.distributed.session.core	解决分布式 session 的问题
DISTRIBUTED 模块	TransactionCore	starwu.lite.distributed.transaction.core	解决分布式事务的问题
MANAGE 模块	AsyncThreadPool	starwu.lite.manage.async.threadPool	Spring 异步功能的增强
UTIL 模块	SnowflakeIdGenerator	starwu.lite.util	雪花算法的增强
MANAGE 模块	ThreadLocalCore	starwu.lite.manage.threadLocal	ThreadLocal 的增强

5 小结

本 STARWU-LITE 开发框架 V1.0 为基于 Java 的自研开发框架，实现了将一个简陋的项目瞬间变成一个成熟的项目的功能，以 Jar 组件库的形式提供给业务项目集成，简化后端开发流程。

本软件所有核心代码均为独立开发完成。

如果在使用遇到，请联系作者，作者姓名：负学文，联系方式/微信：18612230443。